Warsaw University of Technology
Faculty of Mathematics
and Information Science

# Master's diploma thesis

in the field of study Computer Science and Information Systems
and specialisation Artificial Intelligence

## Multi-model LLM system with dynamic planning and model selection optimization

Jakub Knyspel

student record book number 313282

thesis supervisor
dr hab. Maria Ganzha, prof. nzw.

WARSAW 2026

**Abstract**

Multi-model LLM system with dynamic planning and model selection optimization

TODO

**Keywords:** TODO, ...

**Streszczenie**

Implementacja wielomodelowego systemu LLM z dynamicznym planowaniem i optymalizacją doboru modeli

TODO

**Słowa kluczowe:** TODO, ...

# Contents


**1. Introduction** . . . . . **11**

**2. Related Work** . . . . . **12**

2.1. RouteLLM . . . . . 12

2.2. RoRF - Routing over Random Forests . . . . . 12

**3. System Requirements & Constraints** . . . . . **13**

**4. Description of the System** . . . . . **14**

**5. Scoring** . . . . . **15**

5.1. Dataset Selection . . . . . 15

5.2. Preprocessing & Feature Selection . . . . . 16

5.2.1. Complexity Features . . . . . 16

5.2.2. Domain Scores . . . . . 17

5.2.3. Style Features . . . . . 18

5.2.4. Task Type Features . . . . . 18

5.3. Data Analysis . . . . . 19

5.4. LLM Embedding . . . . . 19

5.5. Prompt-invariant Scoring . . . . . 19

5.5.1. Iterative Models . . . . . 19

5.5.2. Non-iterative Models . . . . . 19

5.6. LLM Embedding . . . . . 21

5.6.1. Metrics . . . . . 21

5.6.2. Features . . . . . 21

5.6.3. DNN-based Embedding . . . . . 22

5.6.4. Tuning-based Embedding . . . . . 22

5.6.5. Attention Embedding . . . . . 22

5.6.6. Prompt-invariant Embedding-based Models . . . . . 22

5.7. Simple Models . . . . . 22

5.7.1. DNN Model . . . . . 22

5.7.2. Gradient Boosting Model . . . 22
5.8. Complex Models . . . 22
5.8.1. Transformer Model . . . 22
5.8.2. Response Predictive Model . . . 23
5.8.3. Multistage Models . . . 23
**6. Response Length Prediction . . . 24**
6.1. Input and Output Data . . . 24
6.2. Models . . . 24
6.2.1. Gradient Boosting Model . . . 24
6.2.2. DNN Model . . . 24
**7. Analysis of the Solution . . . 25**
**8. Conclusion . . . 26**

# 1. Introduction

TODO

# 2. Related Work

## 2.1. RouteLLM

TODO

## 2.2. RoRF - Routing over Random Forests

TODO

TODO: More references

# 3. System Requirements & Constraints

TODO: What did I decide to do? What constraints did I settle on?

# 4. Description of the System

TODO: Description of system components, communication, etc. TODO: Also include model selection based on price and score.

# 5. Scoring

TODO

## 5.1. Dataset Selection

The title of the most popular human-evaluated LLM preference dataset without a doubt belongs to *Chatbot Arena Conversations* [TODO: ref]. It is used as primary human-evaluated dataset in referenced works [TODO: verify and ref].

*Chatbot Arena Conversations* data consists of a pair of responses from different LLM models to a single user prompt, along with a comparison label, determining which model's answer is considered better by the user. When comparison is unclear, the label can also indicate that both responses were sufficient for a given query, or that neither model managed to produce a good answer. Selection of LLMs is sufficiently wide for a model to learn a broad spectrum of differences between different LLM models, while prompts span a variety of topics.

However, there is a crucial issue: published in 2022 [TODO: verify], *Chatbot Arena Conversations* dataset only includes comparisons between rather old LLM models. This isn't a disqualifying problem, as my work doesn't focus on particular LLM models. Though, A decision to use this dataset would require two important implications: that human preference during evaluation of a pair of responses remains unchanged, and that the general distribution of responses produced by LLMs is similar.

It's hard to evaluate how close to the truth the first assumption is. While human preferences likely did not change drastically over the last four years, expectations of what LLMs should be able to do did undergo an undeniable change, simply due to their capabilities increasing. Another significant difference may be caused by a wider adoption - people whose responses were used in *Chatbot Arena Conversations* dataset belonged to a significantly narrower group of "early adopters" of LLM technology, compared to a group of LLM users today.

These reasons prompted me to decide to rely on a newer alternative to *Chatbot Arena Conversations* dataset. Collected and released by the same organization (*LMSYS* [TODO: ref]),

*LMArena Human Preference* is a successor of aforementioned dataset. It's structure closely resembled *Chatbot Arena Conversations*, also including human comparison results over a pair of LLM responses to the same prompt. Additionally, the dataset includes a range of common prompt labels [TODO: describe], which can be used as additional features in any prediction task. *LMArena Human Preference* is also an improvement in terms of size. It contains around 140 thousand comparison pairs (compared to *Chatbot Arena Conversations*'s 33 thousand), and includes responses from 53 different models (compared to *Chatbot Arena Conversations*'s 20).

It's important to note that due to a very similar structure, these datasets can be used interchangeably. Due to previously mentioned reasons, mainly *LMArena Human Preference* was used for model training and evaluation throughout this work. However, comparisons between datasets are made where relevant, highlighting important differences between their data.

## 5.2. Preprocessing & Feature Selection

Intuitively, when it comes to inferring the quality of LLM's response to a given input, the most important aspect of that prompt should be it's semantic meaning - ideally, prompts that are equivalent when it comes to meaning, differing only in wording, should produce the same answer. Semantic meaning of a given text can be captured via a range of specialized text embedding models, which, given an input, return a vector that should be useful for a range of classification or regression tasks. [TODO: references, examples]. As such, prompt embedding vectors are a primary candidate for model input features.

In practice, assumption that prompt features which are unrelated to it's meaning shouldn't influence LLM output isn't reasonable. LLMs in they current state are far from ideal, so certain biases and simple relationships cannot be ruled out. Thus, instead of relying solely on prompt embedding vectors, input features also contain a set of supporting values that represent certain characteristics of submitted question.

Aside from prompt embedding vectors, the following simple features were used:

### 5.2.1. Complexity Features

8 features encoding prompt complexity were defined:

- **Character count**;
- **Word count**;
- **Sentence count**;

- **Average word length**;
- **Question density** - Number of question marks in the prompt, divided by prompt length;
- **Nesting density** - Number of parentheses, divided by prompt length;
- **Multi-part indicator density** - Number of words indicating multiple aspects/parts in the prompt ("and", "also", etc.), divided by prompt length;
- **Inverse vocabulary richness ratio** - $1 - \frac{[\text{number of unique words}]}{[\text{word count}]}$.

Features that don't directly encode prompt length were normalized by that length, in order to reduce dependency between them - for example, it's reasonable to assume that longer prompts will, on average, contain more parentheses and question marks than shorter prompts.

Vocabulary richness ratio was inverted by subtracting it's value from 1. This transformation was performed to make value distribution more similar to other features, which made it possible to unify it's further processing with other features [TODO: ref]. In an overwhelming majority of samples, nearly all words are different, causing vocabulary richness ratio to primarily take a value of 1 (or close to it). "Inverting" values causes them to instead be 0 (or close to it).

### 5.2.2. Domain Scores

12 features describing prompt domain were defined:

- **Science**;
- **Medicine**;
- **Law**;
- **Finance**;
- **Tech**;
- **Academic**;
- **Casual**;
- **Formal**;
- **Philosophical**;
- **Historical**;
- **Personal**;

- **Business**;

Each domain score was defined as a ratio of words related to that domain, to total word count of the prompt.

### 5.2.3. Style Features

6 style features were defined:

- **Informal marker density** - Number of informal style indicators (like "btw", "tbh", etc.), divided by prompt length;
- **Specificity marker density** - Number of words indicating constraints and requirements (like "exactly", "must", etc.), divided by prompt length;
- **Politeness marker density** - Number of words indicating polite tone (like "please"), divided by prompt length;
- **Urgency marker density** - Number of words indicating urgency (like "quickly", "fast", etc.), divided by prompt length;
- **Starts with verb** - Boolean feature indicating that first word of the prompt is a verb;
- **Is a question** - Boolean feature indicating that the prompt ends with a question mark;

### 5.2.4. Task Type Features

10 boolean features defining type of the task given by a prompt were defined:

- **Expects code** - Whether prompt contains words indicating that code should be returned;
- **Contains code** - Whether prompt contains code blocks;
- **Expects math** - Whether prompt contains words indicating that some mathematical calculation should be performed;
- **Contains numbers** - Whether prompt contains numbers;
- **Contains math symbols** - Whether prompt contains common math symbols (i.e. equality symbol);
- TODO: Continue

## 5.3. Data Analysis

TODO

## 5.4. LLM Embedding

TODO

## 5.5. Prompt-invariant Scoring

A natural simplification of prompt routing problem comes from an assumption that LLM performance does not depend on the prompt. In this case, every LLM has a single score associated with them, and the problem reduces to assigning those scores based on comparison data. While this simplification makes the problem much less interesting and complex, it is more commonly used when ranking LLMs, due to capturing a general LLM quality rather than focusing on particular cases [TODO: examples, refs]. Also, exploration of this problem will serve as a good baseline for prompt-aware scoring.

### 5.5.1. Iterative Models

TODO (do I need this section?)

### 5.5.2. Non-iterative Models

The problem of scoring entries based on comparison data can be expressed more rigorously, turning it into an optimization problem with well-defined fitness metric. Some such expressions have easily computable optimal solution.

#### L1 Criterion

The most natural way of expressing quality of score assignment is via accuracy - a percentage of comparisons where winning entry was assigned a higher score than the losing one. This is equivalent to minimizing the number of *upsets* (comparisons such that entry A wins against entry B, but A was assigned a lower score than B). This criterion turns out to be NP-hard [TODO: ref]. However, if we take into account the magnitude of upsets (difference between scores assigned to A and B), the problem becomes solvable in polynomial time. Minimizing the sum of magnitudes

can be expressed as a linear programming problem ($v_a$ represents the score assigned to entry A, and $\epsilon$ is added to prevent trivial solution of all scores being 0):

$$\begin{aligned} \text{minimize} \quad & \sum s_i \\ \text{subject to} \quad & v_a - v_b + s_i \geq \epsilon \\ \text{for} \quad & a > b \quad \text{in comparison set} \end{aligned}$$

This criterion can be equivalently formulated via a min-cost-max-flow problem.

1. Each node (except source and sink) represents an entry;
2. A comparison in which A wins against B is expressed as an edge from node A to node B, of capacity 1 and cost 1;
3. There's an edge from source to each node, with cost 0 and capacity equal to it's number of wins;
4. There's an edge from each node to sink, with cost 0 and capacity equal to it's number of losses.

Then, node potentials can be interpreted entry scores.

[TODO: results]

The solution to min-cost-max-flow problem also gives us an additional useful metric: percentage of "meaningful" comparisons in the comparison set. Any cycle in the network has positive cost (since all edges that don't include source and sink have positive cost), so optimal flow won't use them. Using that information, we can compute the ratio of used comparison edges (equal to total cost) to all comparisons. This quantity can be interpreted as decisiveness of comparison set - since comparison cycles can't be scored correctly, the more common they are, the less correct is it to represent LLM quality via a single, prompt-independent score.

### L2 Criterion

Minimizing squares of upset magnitudes turns out to be even simpler - it is a variation of "least squares" problem. As such, this approach was chosen.

[TODO: results]

Turns out that this criterion results in a significantly better accuracy than linear one. As such, it's accuracy will be used as baseline for other models.

## 5.6. LLM Embedding

LLM ecosystem is constantly evolving, with new models being introduced often. While dataset selection was made with that in mind, and chosen dataset contains comparisons of LLMs that are quite new at the time of writing, constraining scoring models to only work with LLMs featured in a comparison dataset doesn't seem like a practical decision. Such datasets rely on human evaluation, and aren't easy to prepare.

An extensible alternative to scoring only LLMs whose comparison data is available, is to compute an embedding vector representing each model, and use that instead of LLM ID as scoring input data. Such embeddings can be computed without relying on comparison data, making the model easily extensible. In particular, embeddings can be computed from a set of prompts and LLM responses to them. This information is much easier to gather than human evaluations, and can be collected automatically.

An argument can also be made that prompts and responses represent all information required for scoring. This perspective makes it possible to define embedding computation problem in a way that avoids underspecification problem [TODO: better name?]. Comparisons dataset that was selected contains promtps and responses from 50 distinct LLMs, which severely limits the dimensionality of embedding vectors. By viewing LLM characteristics as only defined by responses they produce to given prompts, we can instead focus on computing embedding vectors for each prompt and response pair, subsequently defining LLM embeddings as a mean (or in general: some function) of embeddings computed from their prompt and response pairs.

### 5.6.1. Metrics

Embeddings will be used by scoring models to distinguish between LLMs, and subsequently be able to determine certain characteristics of their response patterns. This means that the most important quality of LLM embeddings is distinguishability - given a prompt and response pair, we should be able to tell which LLM it was produced by. Rigorously, we can define an accuracy metric as follows:

TODO: equation

TODO: Other metrics?

### 5.6.2. Features

As described in section 5.2, text embedding vectors are used as input features. As LLM embeddings are computed from both prompts and responses, text embedding vectors of both are

included in input data. Additionally, certain handcrafted response features are also included.

[TODO: list features]

#### 5.6.3. DNN-based Embedding

The simplest implementation of embedding model is simply a multi-layer perceptron. *LeakyReLU* activation function was used, and dropout layers were introduced to prevent overfitting.

Model was trained using triplet margin loss, given by formula [TODO: ref]. Loss criterion can be described as a requirement that a pair of samples from the same LLM should be closer to each other than to a third sample, produced by a different LLM.

#### 5.6.4. Tuning-based Embedding

TODO

#### 5.6.5. Attention Embedding

TODO

#### 5.6.6. Prompt-invariant Embedding-based Models

TODO

### 5.7. Simple Models

#### 5.7.1. DNN Model

TODO

#### 5.7.2. Gradient Boosting Model

TODO

### 5.8. Complex Models

#### 5.8.1. Transformer Model

TODO

### 5.8.2. Response Predictive Model

TODO

### 5.8.3. Multistage Models

TODO

# 6. Response Length Prediction

TODO: Should it be a separate chapter?

## 6.1. Input and Output Data

TODO

## 6.2. Models

### 6.2.1. Gradient Boosting Model

TODO

### 6.2.2. DNN Model

TODO

# 7. Analysis of the Solution

TODO: Tests of all parts of the system, etc.

# 8. Conclusion

TODO

## List of symbols and abbreviations

| | |
|---|---|
| nzw. | nadzwyczajny |
| * | star operator |
| ~ | tilde |

If you don't need it, delete it.

# List of Figures

If you don't need it, delete it.

# Spis tabel

If you don't need it, delete it.

## List of appendices

1. Appendix 1
2. Appendix 2
3. In case of no appendices, delete this part.